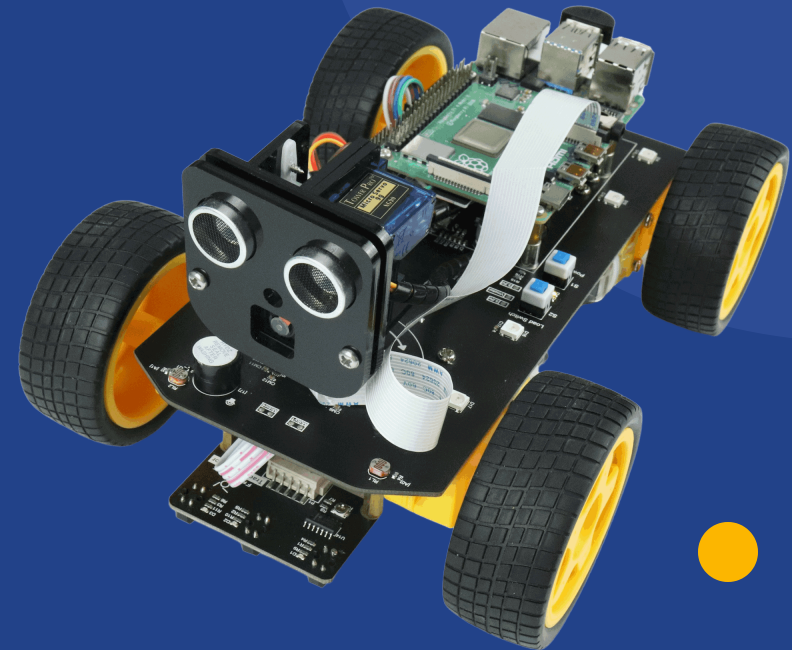




LESSON 3 – ADC & MULTITHREADING

Machine Learning with Raspberry Pi & Smart Car

Today the car reads its own sensors – and watches its battery in the background while it does.



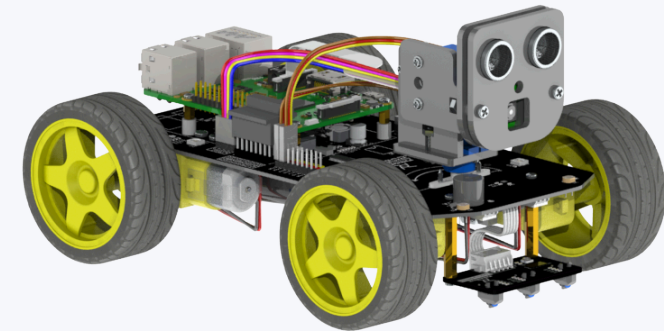


LESSON 3 – ADC & MULTITHREADING

From driving to sensing

Last time the car moved. Today it starts to notice the world around it.

- **Last time:** drove the wheels, beeped the buzzer, and built **command strings** like `CMD_MOTOR#...`.
- **We kept the shape:** `setup() → work → destroy()`, with `import car_setup` first.
- **Today:** the car **reads its own sensors** and **watches its battery** in the background.



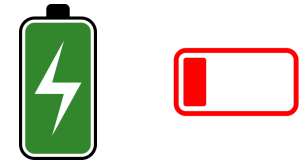


LESSON 3 – ADC & MULTITHREADING

Today's goal: two loops at once

One program that prints the light sensors while a battery watchdog runs alongside it.

- 1 Keeps printing the **two light sensors** – left and right
- 2 Runs a **battery watchdog** that beeps when power gets low
- ★ Both loops run **at the same time** – that's multithreading



"At the same time" is the new idea – not just reading sensors.



LESSON 3 – ADC & MULTITHREADING

What is an ADC?

It turns a smooth analog voltage into a number the computer can read.

- Sensors like light and battery give a smooth **analog voltage**.
- But computers only understand **digital numbers**.
- The **ADC** – Analog-to-Digital Converter – translates voltage into a number.
- We read it with `adc.readADC(channel)`, which returns **volts**.

ANALOG IN

2.5 V

a smooth voltage



ADC

NUMBER OUT

2.5

your code can use

LIKE A DIMMER SWITCH

smooth in → a number you can read



LESSON 3 – ADC & MULTITHREADING

The sensors we'll read

The ADC has numbered inputs called **channels**. "Read a sensor" means "read its channel number."

0 Left light

The left photoresistor – brighter light, different voltage.

1 Right light

The right photoresistor – the partner to channel 0.

2 Battery

The power pack – this reading $\times 3$ is the real voltage.



Find the **two small light sensors** on your board – they're the channel-0 and channel-1 inputs.



LESSON 3 – ADC & MULTITHREADING

Battery math

The ADC only sees one third of the pack voltage – so we multiply the reading by 3.

- The ADC sees **one third** of the battery → multiply by **3**.
- Healthy is about **8 V**. Below that, we warn.
- Rough percent: map **7.0 V → 0%** and **8.4 V → 100%**.
- Your phone estimates its battery the same way – voltage to percent.

**≈ 8 V – healthy**

no warning

**7.0 V or lower – low**beep **twice****6.8 V or lower – critical**beep **four times** – charge now



LESSON 3 – ADC & MULTITHREADING

What is multithreading?

A thread is a separate line of work that runs at the same time as the rest.

- Today the **main thread** prints the light readings.
- A **second thread** watches the battery in the background.
- Both happen at once – **neither waits** for the other.
- Like cooking dinner while a timer quietly watches the oven.

MAIN THREAD

print light

print light

print light

BATTERY THREAD

check battery

check battery

↓ **time** → both columns advance together



LESSON 3 – ADC & MULTITHREADING

Daemons and a stop flag

Two small ideas keep our background thread tidy – and let us shut it down cleanly.

DAEMON THREAD

Quits with the program

A daemon thread is allowed to stop automatically when the main program ends, so it never holds the program open.

EVENT

A simple stop switch

An `Event` is an on/off flag. We flip it to tell the thread "time to stop" – a clean shutdown.

WHY IT MATTERS A clean shutdown means we never leave a thread running or the ADC half-open.



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 1

Start the file: imports

`car_setup` first, then the threading tools and the parts we read.

- `import car_setup` first – it finds the car's modules.
- `Thread` and `Event` come from Python's built-in `threading`.
- Then the **ADC** (sensors) and the **buzzer** (the low-battery beep).

```
lesson_3_adc_multithreading.py

"""Lesson 3: Multithreading and the ADC."""

import car_setup      # add the car's code folders
import time

# a thread runs alongside the rest; an Event is a stop flag
from threading import Thread, Event

from adc import Adc      # reads sensor voltages
from buzzer import Buzzer # the low-battery beeper
```



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 2

Settings: the ADC channels

Give the channel numbers friendly names so the code reads clearly.

- Naming the channels beats scattering bare `0`, `1`, `2` through the code.
- Now `recvADC(BATTERY)` reads like a sentence.
- The comment reminds us of the battery **× 3**.

```
lesson_3_adc_multithreading.py

# ---- ADC channels ----
LEFT_LIGHT = 0
RIGHT_LIGHT = 1
BATTERY = 2           # reading × 3 = real pack voltage
```



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 3

Settings: battery thresholds

Two warning levels, in volts – the numbers you'll tweak later.

- `LOW_BATTERY` = the first warning – beep twice.
- `CRITICAL_BATTERY` = time to charge – beep four times.
- Both are in **volts**, after the $\times 3$ scaling.

```
lesson_3_adc_multithreading.py

# ---- battery settings (volts) ----
LOW_BATTERY = 7.0      # beep twice as a warning
CRITICAL_BATTERY = 6.8 # beep four times, time to charge
```



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 4

Empty boxes and the stop flag

Parts start as `None`; we also create the `stop_event` flag now.

- Name the parts now (`None`), build them in `setup()`.
- `Event()` starts **not set**; calling `.set()` later means "stop."
- The flag exists **before** the thread that reads it – order matters.

```
lesson_3_adc_multithreading.py

# ---- car parts (created in setup) ----
adc = None
buzzer = None
stop_event = Event()      # set() it to stop the thread
battery_thread = None
```



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 5

setup(): build and start the thread

Three steps for a thread: make it, set it as a daemon, then start it.

- Create the sensors, then **make and start** the background thread.
- `target=monitor_battery` = "run that function in the thread."
- `daemon = True` = quit with the program. `.start()` launches it.

```
lesson_3_adc_multithreading.py

def setup():
    """Create sensors and start the battery thread."""
    global adc, buzzer, battery_thread
    adc = Adc()
    buzzer = Buzzer()

    battery_thread = Thread(target=monitor_battery)
    battery_thread.daemon = True    # stop with the program
    battery_thread.start()         # begin running now
```



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 6

Read and scale the battery

Read channel 2, multiply by 3, then turn the voltage into a rough percent.

- `recvADC(BATTERY)` gives volts; `× 3` is the real pack voltage.
- `int(...)` drops the decimals to a whole percent.
- Maps `7.0 V → 0%` and `8.4 V → 100%`.

```
lesson_3_adc_multithreading.py

def read_battery_voltage():
    """Scale the ADC reading to the real pack voltage."""
    return adc.recvADC(BATTERY) * 3    # ADC sees 1/3

def battery_percent(voltage):
    """Turn a voltage into a rough 0–100% estimate."""
    return int((voltage - 7) / 1.40 * 100)    # 7.0=0% 8.4=100%
```



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 7

beep(): a reusable helper

Beep on and off a chosen number of times – reusing the buzzer strings from Lesson 2.

- `buzzer.run("1")` turns it on, `"0"` turns it off – same as Lesson 2.
- `_` is a throwaway name – we just want to **repeat**.
- Pass `2` for low, `4` for critical.

```
lesson_3_adc_multithreading.py

def beep(times):
    """Beep on and off a chosen number of times."""
    for _ in range(times): # "_" = a name we don't use
        buzzer.run("1")    # on
        time.sleep(0.1)
        buzzer.run("0")    # off
        time.sleep(0.1)
```



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 8

monitor_battery(): the background loop

This is the function the second thread runs – it loops until told to stop.

- `while not stop_event.is_set()` = loop until told to quit.
- Beeps **more** the lower it gets: critical → 4, low → 2.
- Waits **3** seconds between checks.

```
lesson_3_adc_multithreading.py

def monitor_battery():
    """Runs in the background thread."""
    while not stop_event.is_set():          # loop until stopped
        voltage = read_battery_voltage()
        print("Battery:", round(voltage, 2), "V")
        if voltage <= CRITICAL_BATTERY:
            beep(4)
        elif voltage <= LOW_BATTERY:
            beep(2)
        time.sleep(3)                       # check again in 3 s
```




LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 9

loop(): the main thread

While the battery thread runs quietly, the main thread keeps printing the light sensors.

- Reads **both** light channels every second.
- `round(value, 2)` keeps the printout readable.
- Now there are **two** `while` loops alive at once – multithreading in action.

```
lesson_3_adc_multithreading.py

def loop():
    """The MAIN thread: print the two light readings."""
    while True:
        left = adc.recvADC(LEFT_LIGHT)
        right = adc.recvADC(RIGHT_LIGHT)
        print("Light left:", round(left, 2),
              "right:", round(right, 2))
        time.sleep(1)
```



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 10

Clean up: destroy()

Flip the stop flag, wait for the thread to finish, then close the ADC.

- `stop_event.set()` tells `monitor_battery()` to finish.
- `join(timeout=2)` = wait up to 2 s so we never hang forever.
- `i2cClose()` closes the connection to the ADC chip.

```
lesson_3_adc_multithreading.py

def destroy():
    """Stop the thread and close the sensors cleanly."""
    stop_event.set()          # tell the thread to finish
    if battery_thread is not None:
        battery_thread.join(timeout=2)  # wait (max 2 s)
    if buzzer is not None:
        buzzer.run("0")      # make sure it's off
    if adc is not None:
        adc.i2cClose()       # close the ADC connection
```



LESSON 3 – ADC & MULTITHREADING • THE FILE, CHUNK 11

The main block

The same trusty pattern: `try` runs the work, `finally` always cleans up.

- `try` runs `setup()` then `loop()`.
- `except KeyboardInterrupt` catches **Ctrl+C**.
- `finally` guarantees `destroy()` runs no matter what.
- Third lesson in a row with this block – you should recognise it instantly.

lesson_3_adc_multithreading.py

```
if __name__ == "__main__":  
    try:  
        setup()  
        loop()  
    except KeyboardInterrupt:  
        pass  
    finally:  
        destroy()
```



LESSON 3 – ADC & MULTITHREADING

You built the whole file

One program, two threads running together.

```
imports → channels + thresholds → stop_event → setup() →  
battery helpers → beep() → monitor_battery() → loop() →  
destroy() → main block
```

The two **teal** pieces are the functions that loop – they run **at the same time**.



Reads sensors and watches the battery, both at once.



LESSON 3 – ADC & MULTITHREADING

Run it

Run it from the Pi's terminal – the output interleaves the two threads.

- Light prints every `1 s`, battery every `3 s` – they **mix**.
- The mixed output is the proof: **both threads run at once**.
- Press `Ctrl+C` to stop. Use `sudo` for the run command.

● ● ● Pi terminal

```
cd Smart-Car-Level-3/Code/User
sudo python3 lesson_3_adc_multithreading.py
```

● ● ● output

```
Light left: 1.85 V   right: 1.90 V
Battery: 8.05 V ( 75 % )
Light left: 1.84 V   right: 1.88 V
Light left: 1.86 V   right: 1.91 V
Battery: 8.04 V ( 74 % )
```



LESSON 3 – ADC & MULTITHREADING

Try it yourself

Poke at the sensors and the settings, then watch the numbers change.

- **Cover one light sensor** with your hand and watch its voltage change.
- Raise `LOW_BATTERY` to `7.8` to force a warning beep – then set it back.
- Change the battery check interval from `3` seconds to `5`.

CHALLENGE

Which side is brighter?

Print `"LEFT brighter"` or `"RIGHT brighter"` based on which sensor reads higher – a preview of light-seeking.



LESSON 3 – ADC & MULTITHREADING

Common issues

If something's off, it's almost always one of these.



Weird battery voltage

Remember the **x 3**, and check the ADC wiring.



Won't quit on Ctrl+C

That's the `join(timeout=2)` wait – give it a second.



i2c / OSError

I²C not enabled, or the ADC address / wiring is off.



Light values don't change

Sensor covered already, or wired to the wrong channel.



Check I²C

Most ADC errors come from I²C being disabled. Enable it in `raspi-config` and reseal the ADC.



LESSON 3 – ADC & MULTITHREADING

The car can sense and multitask!

You read analog sensors and ran a background thread you can stop cleanly.

- ✓ Read analog sensors through the ADC – light and battery
- ✓ Ran a background thread and stopped it with an `Event`
- ✓ Touched every core part: lights, sound, motors, commands, sensors

UP NEXT

Lesson 4 Autonomous modes

Put the building blocks together into smarter, more autonomous behavior.



• COACH ONLY • HIDDEN SLIDE

End-of-class checklist

Don't present this slide. Run through it right after class.

- 1 Post the after-class recap**
Summarise what you covered on the class page so students can catch up.
- 2 Log absent students**
Post anyone who missed class in the **#attendance** Discord channel.
- 3 Prep the Pis for next class**
Confirm **I²C** is **enabled** on each Pi and charge the cars before Lesson 4.

EVERY CLASS

- ✓ Post a class announcement
- ✓ Introductions if needed
- ✓ Note absences in #attendance